

Complete Push Notifications Setup Guide

Official Expo Documentation: [Send notifications with the Expo Push Service](#)

Table of Contents

1. [Overview](#)
2. [Prerequisites](#)
3. [Quick Glossary](#)
4. [Initial Setup](#)
5. [Step-by-Step Configuration](#)
6. [Testing Push Notifications](#)
7. [Troubleshooting](#)
8. [File Structure](#)
9. [How It Works](#)

Overview

This project uses **Expo Notifications** to support remote push notifications on iOS and Android. The setup handles:

- Permission requests
- Expo Push Token registration
- Foreground & background notification handling
- Device-specific behavior for iOS and Android
- Local and remote notification support
- A test notification button in Settings

Important Notes:

- **Android SDK 53+:** Remote push notifications do **NOT** work in Expo Go. You must use a development build.
- **iOS:** Remote push works in both Expo Go and development builds (on physical devices).
- **Physical Devices Required:** Push tokens only work on real devices, not simulators/emulators.

Prerequisites

Before starting, ensure you have:

1. **Physical device** (iOS or Android) — push tokens don't work on simulators/emulators
2. **Expo account** — sign up at expo.dev if you don't have one
3. **Node.js and npm** installed
4. **Expo CLI** installed globally (optional, but recommended):

```
npm install -g expo-cli
```

5. **EAS CLI** for building development builds:

```
npm install -g eas-cli
```

Quick Glossary

Understanding these terms will help you troubleshoot:

- **Local notification:** Scheduled on-device (works in Expo Go). Example: the "Send Test Notification" button.
- **Remote push notification:** Sent from a server → delivered via FCM/APNs. Android remote push does **not** work in Expo Go on SDK 53+.
- **Expo Push Token:** What your app gets from Expo (`ExponentPushToken[...]`). You use it to send a push via Expo's push service.
- **FCM (Firebase Cloud Messaging):** Android's push transport (Google Play Services required).
- **APNs (Apple Push Notification service):** iOS's push transport.
- **Development Build:** A custom build of your app with native code compiled. Required for Android remote push on SDK 53+.
- **Expo Go:** The standard Expo app for quick testing. Limited native functionality.

Initial Setup

Step 1: Install Dependencies

Install required packages:

```
npm install
npx expo install expo-notifications expo-device expo-constants
```

Step 2: Verify Project Structure

Ensure your project has the following structure:

```
lab6/
├─ hooks/
│   └─ usePushNotifications.js  ← Push notification hook with device-specific
behavior
├─ utils/
│   └─ pushNotifications.js    ← Registration & notification helpers
├─ contexts/
│   └─ LanguageContext.js      ← Language context (if using)
├─ App.js                     ← App entry point with push notification hook
└─ Screens/
```

```
| └─ SettingsScreen.js
|   └─ app.json
```

← Push token display + test button
← Expo notifications plugin config

Step-by-Step Configuration

Step 1: Initialize EAS Project

Problem: You may see `"projectId": Invalid uuid` error.

Solution: Initialize EAS project to get a valid project ID.

```
npx eas-cli init
```

What this does:

- Prompts you to log in to EAS (create account if needed)
- Creates/links a real EAS project
- Updates `app.json` with:
 - `expo.extra.eas.projectId` (a UUID)
 - `expo.owner` (your Expo username)

Verify: After running, check `app.json` contains:

```
{
  "expo": {
    "extra": {
      "eas": {
        "projectId": "your-uuid-here"
      }
    },
    "owner": "your-expo-username"
  }
}
```

Step 2: Configure app.json

Ensure your `app.json` has the following configuration:

```
{
  "expo": {
    "name": "Your App Name",
    "slug": "your-app-slug",
    "version": "1.0.0",
    "platforms": ["ios", "android"],
    "ios": {
```

```
    "bundleIdentifier": "com.unbc.app"
  },
  "android": {
    "package": "com.unbc.app",
    "googleServicesFile": "./google-services.json"
  },
  "plugins": [
    [
      "expo-notifications",
      {
        "icon": "./assets/icon.png",
        "color": "#006400"
      }
    ]
  ],
  "extra": {
    "eas": {
      "projectId": "your-project-id-here"
    }
  },
  "owner": "your-expo-username"
}
```

Key Points:

- `ios.bundleIdentifier`: Required for iOS push notifications
- `android.package`: Required for Android push notifications
- `android.googleServicesFile`: Points to Firebase config (see Step 3)
- `plugins`: Configures expo-notifications with icon and color

Step 3: Firebase Setup for Android

Problem: `Default FirebaseApp is not initialized` error on Android.

Solution: Configure Firebase for Android.

3.1: Create Firebase Project

1. Go to [Firebase Console](#)
2. Create a new project (or use existing)
3. Add an **Android app** to your project

3.2: Configure Android App in Firebase

1. **Package Name:** Must exactly match `app.json` → `android.package`
 - This project uses: `com.unbc.app`
 - ⚠ **Critical:** Package name must match exactly!

2. Download `google-services.json` file
3. Place `google-services.json` in project root (same folder as `app.json`)
4. Update `app.json` (if not already done):

```
{
  "expo": {
    "android": {
      "googleServicesFile": "./google-services.json",
      "package": "com.unbc.app"
    }
  }
}
```

Security Note: `google-services.json` isn't as sensitive as a service account key, but you still generally don't want it in public repos. Add to `.gitignore` if needed.

Step 4: Build Development Build (Android - Required for Remote Push)

Problem: Remote push does not work in Expo Go (SDK 53+).

Warning you might see:

Android Push notifications (remote notifications) functionality provided by expo-notifications was removed from Expo Go with the release of SDK 53. Use a development build instead of Expo Go.

Solution: Build and install a development build APK.

4.1: Configure `eas.json`

Create or update `eas.json` in project root:

```
{
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "android": {
        "buildType": "apk"
      }
    }
  }
}
```

Critical: `"buildType": "apk"` is required! Without it, EAS will generate an AAB file (Android App Bundle) which cannot be easily installed directly to a physical phone for testing.

4.2: Build Development APK

```
npx eas-cli build --profile development --platform android
```

What happens:

- EAS builds your app in the cloud
- You'll get a QR code to download the APK
- Scan QR code with your Android device
- Install the APK on your device

Note: This build process takes 10-20 minutes. The APK will have native code compiled with Firebase configuration.

Step 5: Configure FCM V1 Credentials (Android Remote Push)

Problem: Expo can't deliver pushes - "Unable to retrieve the FCM server key for the recipient's app"

Meaning: Your phone got an Expo push token, but Expo's servers aren't authorized to talk to your Firebase project to deliver Android pushes.

Solution: Upload FCM V1 Service Account Key to Expo.

5.1: Generate Service Account Key

1. Go to [Firebase Console](#)
2. Select your project
3. Go to **Project Settings** → **Service accounts** tab
4. Click **Generate new private key**
5. Download the service account JSON file
 - File name will be something like: `your-project-firebase-adminsdk-xxxxx.json`

5.2: Upload to Expo Dashboard

1. Go to [Expo Dashboard](#)
2. Select your project
3. Go to **Credentials** → **Android**
4. Under **Push Notifications (FCM V1)**, click **Upload**
5. Upload the service account JSON file you downloaded

Security Note: Treat the service account JSON as a secret. Don't commit it to git. Add to `.gitignore`:

```
*firebase-adminsdk*.json
```

Step 6: iOS Configuration (Optional - for iOS remote push)

For iOS remote push notifications:

1. **Apple Developer Account:** Required (paid account)
2. **APNs Certificates:** Configure in Apple Developer Portal
3. **Upload to Expo:** Go to Expo Dashboard → Credentials → iOS → Push Notifications

Note: iOS remote push works in Expo Go, but for production you'll need proper certificates.

Testing Push Notifications

Test 0: Verify Local Notifications Work

Purpose: Verify notification UI and permissions are working.

1. Run `npx expo start` (or `npx expo start --dev-client` for dev build)
2. Open app on physical device
3. Grant notification permissions when prompted
4. Go to **Settings** screen
5. Tap "**Send Test Notification (Local)**"
6. A notification should appear in ~2 seconds ✓

If this fails:

- Check notification permissions in device Settings
 - Verify you're on a physical device (not simulator)
 - Check console for errors
-

Test 1: Get Push Token

1. Open app on physical device
2. Go to **Settings** screen
3. The **Push Token** should appear automatically
4. Copy the token (format: `ExponentPushToken[xxxxxxxxxxxxxxxx]`)

If token is `null`:

- Verify you're on a physical device
 - Check notification permissions are granted
 - For Android: Ensure you're using development build (not Expo Go) for remote push
 - Check console for errors
-

Test 2: Send Remote Push via Expo Tool

Purpose: Test end-to-end remote push delivery.

1. Copy your push token from Settings screen
2. Open browser and go to expo.dev/notifications
3. Paste token in "**To (Expo push token)**" field

- 4. Fill **Message Title** and **Message Body**
- 5. Click **Send Notification**
- 6. Notification should arrive on your device immediately 

Note: Depending on how your code handles foreground notifications, you may need to minimize the app to see the system notification pop-up.

Test 3: Send Remote Push via API

Purpose: Test programmatic push sending (for backend integration).

```
curl -X POST https://exp.host/--/api/v2/push/send \
-H "Content-Type: application/json" \
-d '{
  "to": "ExponentPushToken[your-token-here]",
  "title": "Hello!",
  "body": "This is a remote push notification.",
  "data": {
    "screen": "Home"
  }
}'
```

Response: You'll get a JSON response with delivery status.

Troubleshooting

Quick Troubleshooting Checklist

Issue	Solution
Token is null	Verify you're on a physical device and granted notification permission
Remote push never arrives	You must be using a dev build (not Expo Go) and have FCM V1 configured in Expo
"Must use physical device"	Push tokens only work on real devices, not simulators
No notification appears	Check that permissions are granted in device Settings
Android notifications silent	The default channel is set to MAX importance — check device Do Not Disturb mode
"projectId": Invalid uuid	Run npx eas-cli init to create/link EAS project
"Default FirebaseApp is not initialized"	Ensure google-services.json is in project root and app.json points to it
"Unable to retrieve FCM server key"	Upload FCM V1 Service Account Key in Expo Dashboard → Credentials → Android

Issue	Solution
Huawei / no Play Store device	FCM may not work without Google Play Services
Network issues	Try cellular data or a different network (some public Wi-Fi blocks Firebase ports)
Physical device silently failing	Ensure you rebuild the <code>.apk</code> after adding <code>google-services.json</code>
School WiFi Blocked	Try turning off WiFi and using Cellular Data

Detailed Troubleshooting

Issue: Token is null on Settings screen

Possible causes:

1. Running on simulator/emulator (push tokens don't work)
2. Notification permissions not granted
3. Missing dependencies

Solutions:

1. Use a physical device
2. Check device Settings → Apps → Your App → Notifications (ensure enabled)
3. Reinstall dependencies:

```
npx expo install expo-notifications expo-device expo-constants
```

Issue: Remote push never arrives (Android)

Possible causes:

1. Using Expo Go (SDK 53+ doesn't support remote push)
2. FCM V1 credentials not uploaded
3. Firebase not configured

Solutions:

1. Build and use development build (see Step 4)
2. Upload FCM V1 Service Account Key to Expo Dashboard
3. Ensure `google-services.json` is in project root and `app.json` references it

Issue: Build fails or APK is AAB format

Solution: Ensure `eas.json` has `"buildType": "apk"`:

```
{  
  "build": {
```

```
"development": {
  "android": {
    "buildType": "apk"
  }
}
```

Issue: Notifications work locally but not remotely

Checklist:

- ☒ Using development build (not Expo Go) for Android
- ☒ FCM V1 credentials uploaded to Expo Dashboard
- ☒ Firebase project configured correctly
- ☒ Package name matches between Firebase and `app.json`
- ☒ Rebuilt APK after adding `google-services.json`

File Structure

Key Files Explained

```
lab6/
├── hooks/
│   └── usePushNotifications.js
│       └── Custom hook that:
│           - Configures notification handler with device-specific behavior
│           - Registers for push notifications
│           - Sets up foreground and response listeners
│           - Handles cleanup
├── utils/
│   └── pushNotifications.js
│       └── Utility functions:
│           - registerForPushNotificationsAsync(): Requests permissions, creates
Android channel, returns token
│           - sendTestNotification(): Schedules local test notification
├── App.js
│   └── Initializes usePushNotifications() hook
├── Screens/
│   └── SettingsScreen.js
│       ├── Displays push token and test notification button
│       ├── Shows device/platform info
│       └── Android-specific remote push warnings
└── app.json
    └── Expo configuration:
```

- iOS bundle identifier
- Android package name
- Google Services file path
- Notification plugin config
- EAS project ID

How It Works

1. Notification Handler ([hooks/usePushNotifications.js](#))

The hook configures device-specific behavior:

iOS:

- Shows alert, plays sound, and sets badge for both remote and local notifications

Android:

- Remote notifications: Shows alert and plays sound
- Local notifications: Shows alert, plays sound, and vibrates
- No badge (Android doesn't use badges the same way)

2. Registration ([utils/pushNotifications.js](#))

[registerForPushNotificationsAsync\(\)](#):

1. Checks if device is physical (simulators don't work)
2. Creates Android notification channel (Android 8+)
3. Requests notification permissions
4. Gets Expo Push Token from Expo servers
5. Returns token or null if failed

3. Listeners ([hooks/usePushNotifications.js](#))

Foreground Listener:

- Fires when notification received while app is open
- Logs notification details
- Handles device-specific behavior

Response Listener:

- Fires when user taps on notification
- Can trigger navigation based on notification data
- Logs user interaction

4. Settings Screen ([Screens/SettingsScreen.js](#))

- Displays current push token
- Shows device/platform information

- Android-specific remote push support warnings
 - Test notification button (local)
 - Helpful info messages
-

Platform-Specific Notes

Android

Requirements for Remote Push:

- ☒ Development build (not Expo Go)
- ☒ Physical device
- ☒ Firebase configured ([google-services.json](#))
- ☒ FCM V1 credentials uploaded to Expo
- ☒ Google Play Services installed

Local Notifications:

- ☒ Work in Expo Go
- ☒ Work in development builds
- ☒ Work on physical devices

iOS

Requirements for Remote Push:

- ☒ Physical device
- ☒ Works in Expo Go (for testing)
- ☒ Production requires Apple Developer account and APNs certificates

Local Notifications:

- ☒ Work in Expo Go
 - ☒ Work in development builds
 - ☒ Work on physical devices
-

Best Practices

1. **Always test on physical devices** - Simulators/emulators don't support push tokens
 2. **Use development builds for Android** - Required for remote push on SDK 53+
 3. **Keep credentials secure** - Never commit service account keys or certificates to git
 4. **Test both local and remote** - Verify the full pipeline works
 5. **Handle permissions gracefully** - Users may deny notifications
 6. **Log everything** - Helps with debugging
 7. **Test on both platforms** - iOS and Android behave differently
-

References

- [Send notifications with the Expo Push Service](#) - Official Expo documentation for sending push notifications
 - [Expo Notifications Docs](#) - Complete Expo Notifications overview
 - [Expo Push Tool](#) - Web interface for testing push notifications
 - [Expo Push API](#) - API reference for sending notifications
 - [EAS Build Documentation](#) - Building development and production apps
 - [Firebase Console](#) - Firebase project management
 - [Expo Dashboard](#) - Expo project management and credentials
-

Summary Checklist

Before testing remote push notifications, ensure:

- ☐ EAS project initialized (`npx eas-cli init`)
 - ☐ `app.json` configured with correct package/bundle IDs
 - ☐ Firebase project created
 - ☐ `google-services.json` downloaded and placed in project root
 - ☐ `app.json` points to `google-services.json`
 - ☐ `eas.json` configured with `"buildType": "apk"`
 - ☐ Development build created and installed (`npx eas-cli build`)
 - ☐ FCM V1 Service Account Key uploaded to Expo Dashboard
 - ☐ Running on physical device
 - ☐ Notification permissions granted
 - ☐ Using development build (not Expo Go) for Android remote push
-

Last Updated: Based on Expo SDK 54 **Platform Support:** iOS and Android **Remote Push:** Android requires development build, iOS works in Expo Go